

Đề xuất một cơ chế đồng thuận tốc độ cao mới cho blockchain

Tóm tắt điều hành

Trong ngữ nghĩa kỹ thuật nghiêm ngặt, “cơ chế đồng thuận” không chỉ là PoW hay PoS, mà là toàn bộ ngăn xếp gồm chống Sybil, chọn nhánh, finality, lan truyền dữ liệu, và khuyến khích kinh tế để một tập nút phân tán cùng đồng ý về trạng thái sổ cái. Chính vì vậy, khi đánh giá hay thiết kế một consensus mới, chỉ tối ưu riêng người đề xuất khối hoặc riêng fork-choice là chưa đủ. [1]

Những họ giao thức hiện có tối ưu các trục rất khác nhau: PoW kiểu Bitcoin[2] cho tính mở và an ninh permissionless đã được kiểm nghiệm nhưng phải chấp nhận finality chậm và tiêu thụ năng lượng lớn; BFT kiểu PBFT/Tendermint/HotStuff cho finality xác định và độ trễ thấp nhưng chi phí truyền thông/quản trị tăng nhanh theo số validator; các kiến trúc DAG hiện đại như Narwhal/Tusk/Bullshark cho thấy nút thắt thực tế thường nằm ở truyền bá dữ liệu chứ không phải riêng bước bỏ phiếu; còn hybrid như Casper/Gasper hoặc Prism cố gắng ghép những ưu điểm đó lại với nhau, đổi lấy độ phức tạp hệ thống cao hơn. [3]

Từ những quan sát này, báo cáo đề xuất một giao thức mới ở mức kiến trúc, đặt tên là **LUA-DAG** (*Low-latency Unified Availability over DAG*): một cơ chế PoS, permissionless, tách ba việc thường bị buộc chung trong cùng một “block consensus” — **sẵn sàng dữ liệu, sắp thứ tự nhanh**, và **hard finality** — thành ba lớp riêng nhưng liên kết: DAG lan truyền dữ liệu/availability; micro-ordering bằng committee ngẫu nhiên để cho soft-confirmation rất nhanh; và macro-finality bằng quorum stake toàn cục với slashing/accountable safety. Cách ghép này được lấy cảm hứng từ Narwhal/Bullshark cho mặt phẳng dữ liệu, từ Praos/Algorand cho VRF sortition, và từ HotStuff/Casper/Gasper cho finality có trách nhiệm giải trình. [4]

Do yêu cầu đầu vào chưa chốt TPS, độ trễ, ngưỡng chịu lỗi Byzantine, và mô hình mạng, đề xuất này được **tham số hóa**. Tuy nhiên, để giữ một nền tảng có thể chứng minh được, báo cáo chọn baseline **partial synchrony** sau GST và **hard-finality an toàn khi stake Byzantine $< 1/3$** ; muốn đẩy ngưỡng lên gần $1/2$ thì phải thay đổi đáng kể giả định mạng hay mô hình tin cậy. Đây không phải né tránh thiết kế, mà là tôn trọng các cận impossibility và các ngưỡng đã được thiết lập trong lý thuyết đồng thuận. [5]

Bối cảnh và hiện trạng

Khung khái niệm

Một điểm quan trọng thường bị bỏ qua là PoW và PoS tự chúng không phải toàn bộ consensus; chúng chủ yếu là cơ chế chống Sybil và chọn người tác giả khối, còn một blockchain hoàn chỉnh vẫn cần fork-choice, finality và incentive layer. Tài liệu kỹ thuật hiện đại của Ethereum[6] diễn đạt điều này rất rõ: “consensus mechanism” là toàn bộ stack của protocol, incentive và chain-selection, chứ không chỉ là proof-of-stake hay proof-of-work. [7]

Ghi chú bạn tải lên cũng tách đúng không gian thiết kế thành năm họ lớn — PoW, PoS, BFT, DAG và hybrid — điều phù hợp với các survey học thuật gần đây. Đồng thời, bản dịch tiếng Việt chính thức của whitepaper Bitcoin vẫn là một nguồn tham chiếu hữu ích cho lập luận nền tảng về “chuỗi dài nhất” và “đa số CPU”. [\[8\]](#)

So sánh các họ cơ chế đồng thuận

Họ cơ chế	Đại diện điển hình	Độ trễ và thông lượng đại diện	Finality	Giả định an ninh	Phi tập trung	Nhận xét ngắn
PoW	Bitcoin/Nakamoto	Khoảng 10 phút cho block đầu tiên; Bitcoin cổ điển đạt khoảng 3.3–7 TPS trong cấu hình gốc	Xác suất; thực tế thường chờ 6 confirmations, khoảng 1 giờ	Đa số hashpower trung thực, thường diễn đạt là >50% hashpower không bị kẻ tấn công kiểm soát	Mở, permissionless mạnh	Cực kỳ battle-tested, nhưng finality chậm và tiêu tốn điện
PoS	Ouroboros, Praos, Gasper, Algorand	Biến thiên rất lớn; Ethereum hiện có slot 12 giây và finality khoảng 15 phút; Algorand paper nhắm minute-scale confirmation và thông lượng tăng mạnh so với Bitcoin	Kinh tế/accountable hoặc xác suất tùy biến thể	Đa số stake trung thực hoặc siêu đa số 2/3 cho checkpoint/finality	Trung bình đến cao	Tiết kiệm năng lượng, có slashing, nhưng dễ gặp weak subjectivity và tập trung stake
BFT	PBFT,	Quyết	Xác định	Thường $n > 3f$,	Thấp hơn	Rất tốt cho

Họ cơ chế	Đại diện điển hình	Độ trễ và thông lượng đại diện	Finality	Giả định an ninh	Phi tập trung	Nhận xét ngắn
	Tendermint, HotStuff	định trong vài vòng thông điệp; độ trễ thấp, nhưng TPS phụ thuộc mạnh vào kích thước committee và đường dữ liệu		quorum $2f+1$	permissionless PoW/PoS toàn cục	finality nhanh, nhưng khó mở rộng committee lớn
DAG	DAG-Rider, Narwhal/Tusk, Bullshark	Narwhal- HotStuff >130k TPS ở dưới 2 giây; Tusk ~160k TPS ở khoảng 3 giây; Bullshark ~125k TPS ở 2 giây trên 50 nút/party trong benchmark WAN công bố	Xác định hoặc xác suất tùy overlay	Thường <1/3 Byzantine; một số giao thức hỗ trợ asynchronous liveness	Trung bình	Cực mạnh về throughput, nhưng phức tạp hơn về GC, fairness, light clients
Hybrid	Casper/Gasper, Prism	Gasper hiện finality ~15 phút; Prism về lý thuyết	Accountable hoặc xác suất tùy lớp lại	Hỗn hợp: 2/3 checkpoint vote hoặc 50% hashpower, tùy lớp nền	Linh hoạt	Hấp dẫn về mặt kiến trúc, nhưng implementation rất phức tạp

Họ cơ chế	Đại diện điển hình	Độ trễ và thông lượng đại diện	Finality	Giả định an ninh	Phi tập trung	Nhận xét ngắn
		đạt throughput tới capacity C của mạng và latency tỉ lệ với propagation delay D				

Bảng trên dùng các exemplar và số đại diện sau: hàng PoW dựa trên Nakamoto, Bitcoin developer docs và phân tích scaling của Croman; hàng PoS dựa trên Ouroboros Praos, tài liệu hiện hành của Ethereum và paper Algorand; hàng BFT dựa trên PBFT, Tendermint và HotStuff; hàng DAG dựa trên DAG-Rider, Narwhal/Tusk và Bullshark; hàng hybrid dựa trên Casper/Gasper và Prism. Các số benchmark không thể so “apple-to-apple” tuyệt đối vì khác LAN/WAN, payload, cách batching và có/không có execution layer. [9]

Các mẫu kiến trúc ứng viên cho một consensus tốc độ cao

Mẫu kiến trúc	Điểm mạnh chính	Điểm yếu chính	Đánh giá cho thiết kế mới
Chuỗi BFT thuần kiểu HotStuff/Tendermint	Formal model rõ, deterministic finality, linear view-change (HotStuff)	Leader bottleneck; data dissemination vẫn nằm trên đường găng	Tốt làm lớp finality, chưa đủ tốt làm data plane
Mempool tách riêng + BFT ordering kiểu Narwhal-HotStuff	Chứng minh thực nghiệm rằng tách dissemination khỏi ordering cho bước nhảy bậc thang về throughput	Vẫn còn ordering leader-based; liveness/fairness phụ thuộc ordering overlay	Là baseline kỹ thuật rất mạnh
DAG ordering nhúng fast/slow path kiểu Bullshark/Tusk	Throughput/latency rất cao; có đường tốt dưới synchrony và backbone tốt dưới fault/asynchrony	Khó hơn cho light client semantics, checkpointing và quản trị storage	Là nguồn cảm hứng trực tiếp cho lớp micro- ordering
Finality gadget lai kiểu Casper/Gasper	Accountable safety, slashing, checkpoint semantics rõ ràng cho ứng	Nếu dùng nguyên xi thì finality còn chậm	Là nguồn cảm hứng trực tiếp cho lớp macro-

Mẫu kiến trúc	Điểm mạnh chính	Điểm yếu chính	Đánh giá cho thiết kế mới
PoW tách vai trò kiểu Prism	dụng và cầu nối Phân rẽ rất đẹp về mặt lý thuyết, throughput tiệm cận capacity	Vẫn giữ chi phí năng lượng của PoW	finality Không chọn cho thiết kế gốc vì yêu cầu tiết kiệm năng lượng

Nguồn của bảng chọn ứng viên: HotStuff so với BFT-SMaRt và linear view-change; Narwhal/Tusk về tách data plane; Bullshark về fast-path trong partial synchrony; Gasper/Casper về accountable finality; Prism về phân rẽ proposer/voter/transaction blocks và giới hạn vật lý của throughput. [10]

Khoảng trống hiện nay và ràng buộc mục tiêu

Những khoảng trống kỹ thuật chưa được giải quyết tốt

Khoảng trống lớn nhất của consensus hiện đại không còn đơn thuần là “làm sao bỏ phiếu nhanh hơn”, mà là cách **ghép ba mục tiêu xung đột**: finality nhanh, overhead thấp trên mỗi node, và phi tập trung đủ sâu để không đòi hỏi phần cứng hay băng thông quá đắt. Tài liệu nghiên cứu và tài liệu roadmap của Ethereum cùng nhấn mạnh rằng thời gian finality ngắn hơn đòi hỏi xử lý nhiều attestations hơn trong thời gian ngắn hơn; càng nhiều validator, nút càng phải xử lý nhiều hơn; và đây chính là nơi xảy ra trade-off giữa thời gian, overhead và decentralization. [11]

Khoảng trống thứ hai là **đường dữ liệu**. Narwhal/Tusk cho thấy trong nhiều hệ thống ledger quy mô lớn, nút thắt không phải bản thân consensus logic mà là reliable dissemination của transaction data; khi tách dissemination khỏi ordering, HotStuff từ baseline hẹp có thể nhảy vọt lên hơn 130k TPS ở dưới 2 giây trong benchmark tương ứng, còn Tusk và Bullshark tiếp tục đẩy biên đó lên thêm. Điều này cho thấy những thiết kế còn để dữ liệu “đi cùng phiếu bầu” thường sẽ thua trên throughput trước khi chạm trần lý thuyết của phần ordering. [12]

Khoảng trống thứ ba là **finality đủ nhanh nhưng vẫn có semantics mạnh cho cầu nối, light clients và ứng dụng tài chính**. Gasper của Ethereum cho finality kinh tế mạnh, nhưng finality hiện nay vẫn khoảng 15 phút; chính roadmap của Ethereum xem đây là quá dài cho nhiều ứng dụng và chỉ ra rằng độ trễ này mở ra cửa sổ cho short reorg và MEV. Nói cách khác, “head nhanh” nhưng “hard-finality chậm” vẫn là vấn đề thật, chứ không phải chi tiết UX nhỏ. [13]

Khoảng trống thứ tư là **đồng bộ trạng thái và light clients**. Full node vẫn là cách trustless nhất để dùng blockchain, nhưng tài nguyên nhớ/đĩa/CPU khiến không phải ai cũng vận hành được; light client vì vậy là một phần cốt lõi của thiết kế consensus hiện đại chứ không phải phụ kiện. Ethereum đã phải đưa hẳn sync committees vào protocol để light clients có thể theo kịp chain mà không phải xác minh toàn bộ validator set. [14]

Mục tiêu và ràng buộc đầu vào

Yêu cầu đầu vào của bài toán để trống nhiều tham số quan trọng. Vì vậy, thiết kế dưới đây coi các biến này là **tham số cấu hình**, không đóng đinh một profile duy nhất ngay từ đầu.

Mục tiêu hoặc ràng buộc	Trạng thái đầu vào	Quyết định trong báo cáo
Mục tiêu throughput TPS	Chưa chỉ định	Thiết kế phải scale theo committee size, batch size, và băng thông; benchmark sẽ quét tham số thay vì tối ưu cho một con số duy nhất
Mục tiêu độ trễ	Chưa chỉ định	Tách soft-confirmation và hard-finality để không buộc một chỉ số độ trễ duy nhất cho mọi use case
Ngưỡng an ninh Byzantine	Chưa chỉ định	Chọn baseline hard-finality an toàn dưới $< 1/3$ stake Byzantine; các profile cao hơn đòi hỏi giả định mạnh hơn
Hiệu quả năng lượng	Bắt buộc	Loại PoW khỏi thiết kế gốc; dùng PoS với incentive/slashing lượng
Phi tập trung	Bắt buộc	Giữ permissionless membership, VRF committee rotation, light-client semantics gọn
Mô hình mạng	Chưa chỉ định	Chọn partial synchrony + availability path không sập khi trước GST mạng xấu

Lý do chọn baseline $< 1/3$ Byzantine không phải bảo thủ vô cớ, mà là vì đây là vùng thiết kế được PBFT/Tendermint/HotStuff và phần lớn protocol BFT/PoS hiện đại hỗ trợ tốt dưới partial synchrony. FLP cho thấy không thể kỳ vọng termination xác định trong môi trường bất đồng bộ hoàn toàn chỉ với một lỗi; DLS chỉ ra cách vượt hạn này bằng partial synchrony, nhưng đồng thời cũng làm lộ ra ngưỡng chịu lỗi gắn chặt với mô hình fault và xác thực. Với bài toán permissionless, quantum-resistant vừa phải và không giả định trusted hardware, $< 1/3$ là ngưỡng thực tế nhất cho hard-finality mạnh. [15]

Từ đó, hướng hợp lý nhất là **không chọn** giữa “DAG throughput cao” và “BFT finality mạnh”, mà **ghép chúng thành hai nhịp**: một nhịp micro để ordering nhanh và một nhịp macro để finality/accountability. Đây chính là khoảng trống mà LUA-DAG nhắm vào. [16]

Đề xuất giao thức mới LUA-DAG

Ý tưởng cốt lõi

LUA-DAG là một thiết kế mới trong phạm vi báo cáo này, không phải tên của một giao thức đã xuất bản. Điểm mới cốt lõi là: **không để cùng một lớp protocol gánh cả ba nhiệm vụ “phổ biến giao dịch”, “chọn thứ tự”, và “chốt finality”**. Thay vào đó, LUA-DAG dùng một DAG availability để xác nhận rằng dữ liệu đã thật sự đi vào mạng; một lớp micro-ordering bằng committee ngẫu nhiên để tạo soft-confirmation nhanh; và một lớp macro-finality stake-wide để tạo hard-finality có slashing. Cách phân lớp này bám sát bằng chứng thực nghiệm của Narwhal/Bullshark rằng

data plane riêng giúp throughput tăng mạnh, đồng thời bám sát tư tưởng của Casper/Gasper rằng finality mạnh nên có checkpoint semantics, slashability và incentive rõ ràng. [16]

Về triết lý, LUA-DAG chấp nhận rằng **soft-confirmation** và **hard-finality** là hai hợp đồng khác nhau với người dùng. Soft-confirmation phục vụ UX, market-making, game, orderflow nội bộ và workloads độ trễ thấp. Hard-finality phục vụ bridge, settlement giá trị cao, thanh lý tài chính, và state-sync/light clients. Ethereum hiện cũng đang đi theo tư duy này khi tách head selection khỏi finality gadget và nghiên cứu single-slot finality như một đường nâng cấp riêng. [17]

Kiến trúc tổng thể

Hình dưới mô tả ba lớp của LUA-DAG: **Availability DAG**, **micro-ordering**, và **macro-finality**. Availability DAG lấy cảm hứng từ Narwhal; VRF private sortition lấy cảm hứng từ Praos/Algorand; còn checkpoint/slashing và light-client header proofs lấy cảm hứng từ HotStuff/Casper/Ethereum light clients. [18]

flowchart LR

```
A[Client gửi giao dịch] --> B[Batching + erasure coding]
B --> C[Availability DAG]
C --> D[Batch đủ điều kiện để ordering]
D --> E[Micro committee chọn frontier]
E --> F[MicroQC - soft confirmation]
F --> G[Macro checkpoint]
G --> H[MacroQC - hard finality]
H --> I[State sync]
H --> J[Light client]
```

Mô hình mạng và mô hình tin cậy

LUA-DAG giả định một tập validator PoS có trọng số stake w_i , tổng stake hoạt động là W . Mạng là **eventually synchronous**: trước thời điểm GST kẻ tấn công có thể trì hoãn, đảo thứ tự và làm nhiễu thông điệp; sau GST, mọi thông điệp giữa các nút đúng đều tới trong giới hạn Δ chưa biết trước. Đây là mô hình partial synchrony kinh điển dùng bởi DLS, Tendermint, HotStuff và hầu hết BFT blockchain thực dụng. [19]

Kẻ tấn công được xem là **adaptive Byzantine** trong biên stake đã định, có thể equivocate, kiểm soát scheduling trước GST, cố DoS collectors, và thao túng mạng ngang hàng; nhưng không phá được hash, chữ ký, VRF, hay BLS aggregation. Để hạn chế adaptive corruption nhắm đúng proposer/committee, LUA-DAG dùng **VRF private sortition**: validator chỉ tiết lộ mình được chọn khi phát thông điệp đầu tiên, giống tinh thần của Praos và Algorand. [20]

Baseline an ninh của giao thức là **hard-finality an toàn nếu stake Byzantine $< 1/3$ và liveness nếu stake online, trung thực $> 2/3$ sau GST**. PoS còn cần weak subjectivity để chặn long-range attacks đối với node mới hoặc node offline lâu; vì vậy LUA-DAG cũng yêu cầu một checkpoint gần đây làm pseudo-genesis khi state-sync từ đầu, tương tự Ethereum. [21]

Cấu trúc dữ liệu

LUA-DAG không đồng thuận trực tiếp trên một “block” đơn khối chứa mọi bytes giao dịch. Nó dùng bốn cấu trúc chính:

Cấu trúc	Vai trò	Nội dung chính
Batch	Đơn vị chứa giao dịch	batch_id, danh sách tx, Merkle root, erasure root, fee summary
Vertex	Đỉnh DAG availability	epoch, round, author, parent_set, batch_refs, chữ ký
MicroCheckpoint	Đơn vị ordering nhanh	slot, parent_macro, frontier_root, ordered_batch_root, collector_id, MicroQC
MacroCheckpoint	Đơn vị hard-finality	height, parent, micro_head, state_root, receipt_root, MacroQC

Cách “đồng thuận trên reference nhỏ thay vì đồng thuận trên bytes giao dịch thô” là một mẫu đã được Prism và Narwhal chứng minh là đặc biệt hữu ích cho throughput, vì consensus khi đó chủ yếu chỉ sắp thứ tự metadata và chứng chỉ, không chờ toàn bộ dữ liệu nặng nhất. [22]

Luồng availability và lan truyền dữ liệu

Client gửi giao dịch tới bất kỳ ingress validator nào. Validator gom giao dịch thành Batch, mã hóa phân mảnh bằng erasure coding, phát tán chunk cùng commitment. Sau đó validator tạo Vertex cho round hiện tại, tham chiếu ít nhất một quorum cha của round trước và đính các batch_refs mới. Một batch được coi là **availability-certified** khi tồn tại đủ cấu trúc DAG để một quorum lớn validator đã thấy và có thể truy xuất batch đó, tức là batch không chỉ “được nhắc tên” mà còn thoát khỏi rủi ro data withholding đơn giản. Đây là phần được mượn về ý tưởng từ Narwhal: đồng thuận không chờ transaction bytes đi qua leader, mà dùng DAG/quorum để chứng minh dữ liệu đã đi vào mạng. [23]

Để giảm rủi ro dữ liệu bị mất sau khi được tham chiếu, validator có nghĩa vụ lưu batch cho tới khi batch đã nằm sâu hơn một ngưỡng GC_{H_gc} bên dưới checkpoint finalized. Một validator đã ký vào bằng chứng availability nhưng không thể phục vụ batch khi challenged sẽ chịu penalty vận hành; equivocation trên commitment hoặc phục vụ dữ liệu sai là slashable khi có bằng chứng mật mã. Ý tưởng “availability trước, ordering sau” ở đây là lối đi thẳng để loại bỏ nút thất leader/data plane phổ biến trong chain BFT đơn khối. [24]

Leader election và micro-ordering

Mỗi epoch e , giao thức tạo randomness beacon R_e từ checkpoint đã finalized trước đó, ví dụ $R_e = H(R_{e-1} || \text{AggSig}(\text{MacroCheckpoint}_{e-1}))$. Từ R_e , mỗi validator tự chạy VRF để biết mình có được chọn vào **micro committee** của slot s hay không. Từ committee đó, một **collector** chính và vài collector dự phòng được xếp hạng theo output VRF nhỏ nhất. Vì vai trò collector chỉ lộ ra khi phát thông điệp, giao thức giảm bề mặt tấn công targeted DoS so với

leader cố định bị biết trước. Đây là lựa chọn được thúc đẩy bởi Praos và Algorand, nơi private sortition là biện pháp chính chống adaptive targeting. [20]

Ở mỗi slot, micro committee lấy một **cutoff round** của DAG, ví dụ $r_{\text{cut}} = \text{current_round} - L_{\text{avail}}$, và chạy một hàm xác định để chọn **frontier**: tập đỉnh hợp lệ, availability-certified, mở rộng từ macro checkpoint đã khóa gần nhất, và tối đa hóa khối lượng dữ liệu/độ phủ nhân quả. Bởi vì frontier được tính thuần xác định từ cùng một tập DAG và cùng một checkpoint gốc, mọi node đúng sẽ rút ra cùng một frontier_root. Collector phát MICRO-PROPOSAL(frontier_root, parent_macro, high_macro_qc); committee trả MICRO-VOTE; collector gom đủ phiếu và tạo MicroQC. Chữ ký quorum được gộp bằng aggregate signatures để giữ message gọn, cùng tinh thần mà HotStuff dùng threshold/signature combining để giảm footprint truyền thông. [25]

MicroQC cho **soft-confirmation**: tx nằm trong closure của frontier_root được coi là “xác nhận nhanh”, nhưng chưa là settled vĩnh viễn. Nếu collector chính lỗi hoặc bị DoS, collector dự phòng có thể tiếp quản sau timeout ngắn; nếu cả fast path không hình thành, slot đó chỉ mất soft-confirmation chứ không phá hard safety của chain. Bài học từ Bullshark ở đây là fast path nên tồn tại cho common-case synchrony nhưng không được khóa toàn hệ thống vào một leader dài hạn. [26]

Macro-finality, fork-choice và quy tắc commit

Sau mỗi cửa sổ W micro-slots, giao thức tạo một MacroCheckpoint tóm lược micro_head, state_root, receipt_root, availability frontier digest, và parent. Toàn bộ validator set — không chỉ committee nhỏ — bỏ MACRO-VOTE trên checkpoint này. Khi thu được tối thiểu $2/3$ W stake, ta có MacroQC. Một checkpoint được **justified** nếu có MacroQC; nó được **finalized** khi có một checkpoint con kế tiếp cũng justified và trở trực tiếp về nó. Về bản chất, đây là accountable two-chain finality kiểu Casper/FFG, áp lên một chain checkpoint thưa hơn và sống trên đầu micro-ordering nhanh hơn. [27]

Fork-choice của LUA-DAG là ba tầng. Một node trước hết neo vào checkpoint finalized cao nhất. Từ đó, nó chọn descendant có MacroQC cao nhất; trong đoạn chưa macro-finalized, nó chọn MicroQC descendant cao nhất; và nếu vẫn hòa, nó dùng deterministic heaviest-frontier rule trên availability DAG. Điều này giữ semantics đơn giản cho ứng dụng: **không có tx nào dưới finalized checkpoint bị revert**, còn các tx chỉ mới có MicroQC thì có thể bị thay bằng một microchain khác trong phạm vi cho phép. Cách tách “head nhanh” khỏi “finality cứng” phù hợp với phân tách LMD-GHOST + Casper trong Gasper, nhưng LUA-DAG dời phần head nhanh sang frontier của DAG thay vì đầu một block tree đơn. [28]

Về commit semantics, có thể viết gọn như sau:

- MicroQC $\rightarrow$ soft-confirmation, có thể rollback trong cửa sổ ngắn.
- MacroQC $\rightarrow$ justification của checkpoint.
- CP_k finalized khi CP_k justified và $CP_{\{k+1\}}$ justified với $\text{parent}(CP_{\{k+1\}}) = CP_k$.

Chính sách này cố ý làm cho “khi nào an toàn để bridge/rút tiền” trở nên rõ ràng: **chỉ khi có hard finality ở macro layer**. Điều này phản ánh bài học của Ethereum rằng protocol nên biểu diễn finality bằng checkpoint semantics rõ ràng thay vì để ứng dụng tự đoán từ độ sâu block. [29]

Khuyến khích kinh tế và chế tài

LUA-DAG cần incentive đủ tinh để không biến availability thành “việc làm không công”. Một phân rã hợp lý là:

- **Phần thưởng cơ sở** cho stake online và đồng bộ.
- **Phần thưởng availability** cho validator tạo vertex đúng hạn và phục vụ batch khi challenged.
- **Phần thưởng micro** cho committee members và collector hình thành MicroQC.
- **Phần thưởng macro** cho phiếu MACRO-VOTE.
- **Phí inclusion** chia cho ingress/batch producer và proposer/collector đã giúp batch vào macro-finality.

Định hướng này bám sát kinh nghiệm PoS hiện đại: Ethereum thưởng cho attestation, block proposal, sync committee và áp phạt cho sai phạm hoặc bỏ lỡ duty. Đặc biệt, slashing nên nhắm vào hành vi có thể chứng minh rõ ràng: ký hai MICRO-VOTE xung đột cho cùng slot; ký hai MACRO-VOTE xung đột; ký checkpoint với state transition sai nếu có fraud proof tái hiện được; hoặc double-proposal rõ ràng. Các lỗi “không online” xử bằng penalty mềm và inactivity leak thay vì slashing cứng hàng loạt. [30]

LUA-DAG cũng nên có **inactivity leak** ở macro layer: nếu chain không finalize trong L_{leak} macro windows, stake của validator offline hoặc liên tục bỏ phiếu sai sẽ bị bào mòn dần cho tới khi tập validator đang hoạt động lấy lại $>2/3$ stake và khôi phục finality. Đây là cùng nguyên lý Ethereum đang dùng để thoát trạng thái mất finality khi có hơn $1/3$ validator offline. [31]

Đồng bộ trạng thái và light clients

State sync của LUA-DAG có ba bước. Node mới trước hết lấy một **weak-subjectivity checkpoint** gần đây từ nhiều nguồn độc lập. Tiếp theo, node tải chuỗi MacroCheckpoint và các MacroQC kể từ checkpoint đó, xác minh aggregate signature và chain linkage. Cuối cùng, node tải snapshot state tại `state_root` tương ứng, kiểm tra Merkle/Verkle proofs cho từng chunk, rồi replay phần microchain nằm trên đỉnh checkpoint đã finalized. Mô hình này cố ý tách “sync an toàn” khỏi “đuổi head nhanh”, tương tự cách Ethereum phân biệt checkpoint sync/weak subjectivity với head-tracking thông thường. [32]

Light clients của LUA-DAG theo dõi **chỉ macro headers** và aggregate signatures; khi cần chứng minh một tx hoặc account state, chúng yêu cầu proof gắn với `receipt_root` hoặc `state_root` của một checkpoint đã finalized. Đây là sự kết hợp của hai dòng ý tưởng: SPV của Bitcoin — chỉ giữ header chain và Merkle proofs — và sync committees của Ethereum — dùng một tập ký nhỏ/aggregate signatures để giảm chi phí xác minh trên thiết bị nhẹ. Nếu muốn tối ưu hơn nữa

cho mobile, LUA-DAG có thể bổ sung một **sync committee luân phiên** ký các macro headers gần đây. [33]

Hình dưới minh họa luồng thông điệp tối thiểu từ giao dịch tới hard finality. [34]

sequenceDiagram

```
participant C as Client
participant I as Ingress validator
participant D as Availability DAG
participant M as Micro committee
participant K as Collector
participant V as Global validators

C->>I: gửi tx
I->>D: phát batch + commitments/chunks
D-->>D: tạo Vertex và chứng minh availability
M->>K: MICRO-VOTE(frontier_root)
K-->>D: MicroQC
D->>V: đề xuất MacroCheckpoint
V->>V: MACRO-VOTE(checkpoint_hash)
V-->>D: MacroQC / finalized checkpoint
```

Phân tích an ninh hình thức

Mô hình đe dọa

Đối thủ của LUA-DAG có thể làm bốn việc: equivocate trên micro hoặc macro layer; withheld dữ liệu; nhắm DoS vào collectors/committee members; và cố tạo fork dài trên các node mới/offline lâu. Ngoài ra, trong môi trường permissionless, đối thủ còn khai thác grinding trên randomness, fee sniping/MEV, spam batch và client monoculture. Những bề mặt tấn công này đều đã xuất hiện ở các họ consensus hiện nay: long-range và weak subjectivity trong PoS; short reorg/MEV dưới finality chậm; leader DoS trong proposer-based protocols; và data withholding khi dissemination không được ràng buộc tốt. [35]

Do đó, baseline security objective của LUA-DAG được chia rõ làm hai lớp:

- **Safety cứng:** không có hai macro-finalized history xung đột trừ khi tồn tại bằng chứng slashable của ít nhất một ngưỡng stake đáng kể.
- **Liveness cứng:** sau GST, nếu stake trung thực online $>2/3$, chuỗi macro-finality tiếp tục tiến.
- **Soft-confirmation:** xác suất rollback nhỏ và điều chỉnh được bằng committee size, nhưng không được quảng bá như finality tuyệt đối.

Cách tách objective như vậy phản ánh đúng thực tế hệ thống: nhiều protocol hiện đại cho head nhanh hơn finality, và những gì cần cho UX không hẳn giống những gì cần cho settlement-grade safety. [17]

Phác thảo chứng minh an toàn và sống

Mệnh đề an toàn của macro layer. Giả sử có hai MacroCheckpoint xung đột cùng được finalized. Mỗi finalized checkpoint cần một chuỗi justification bằng các MacroQC có tối thiểu $2/3$ W stake. Hai quorum $2/3$ W trên hai nhánh xung đột phải giao nhau ở ít nhất $1/3$ W stake. Vì node trung thực không bỏ phiếu cho checkpoint xung đột với khóa hiện hành của mình, nên phần giao này phải chứa validator đã ký thông điệp slashable. Do đó, vi phạm safety kéo theo evidence kinh tế rõ ràng. Đây chính là accountable-safety kiểu Casper/FFG, được chuyển từ block/checkpoint chain đơn sang macro-checkpoint chain của LUA-DAG. [27]

Mệnh đề nhất quán ordering. Nếu hai node đúng có cùng finalized macro parent và cùng tập availability-certified vertices dưới r_cut , hàm chọn frontier xác định sẽ cho cùng $frontier_root$; vì vậy closure transaction do MicroQC đại diện là như nhau. Điều này không phải một “định lý ngoài trời” mà là yêu cầu căn bản của thiết kế: frontier selection phải là hàm thuần xác định trên DAG snapshot. Bài học rút ra từ HotStuff và Gasper là mọi lớp fork-choice/finality mạnh đều cần một đối tượng đầu vào xác định thì mới có thể nâng thành safety proof ở lớp trên. [36]

Mệnh đề sống. Sau GST, broadcast giữa node đúng trở nên Δ -timely; VRF committee sẽ tiếp tục sinh ra các committee có thành phần trung thực; đường collector dự phòng đảm bảo một collector đúng cuối cùng sẽ nhận đủ phiếu MICRO-VOTE; còn ở macro layer, nếu online honest stake $>2/3$ thì luôn tồn tại MacroQC cho checkpoint kế tiếp. Tinh thần chứng minh ở đây bám theo DLS/Tendermint/HotStuff: safety không cần đồng hồ toàn cục trước GST, nhưng liveness đòi hỏi eventual synchrony và cơ chế tiến view/timeout chuẩn. [19]

Các vector tấn công và đối sách

Tấn công	Hậu quả	Đối sách trong LUA-DAG
Long-range PoS fork	Lừa node mới/offline lâu sync vào chain giả	Weak-subjectivity checkpoint, withdrawal delay, không cho reorg trước checkpoint tin cậy
Data withholding	Soft-confirm batch nhưng không ai lấy được data	Chỉ cho ordering trên batch availability-certified; challenge/serve proofs; lưu batch tới quá ngưỡng GC
Collector/leader DoS	Mất liveness ngắn hạn của micro path	Collector xếp hạng bằng VRF, có backup collectors, timeout ngắn và failover
Committee capture	Soft-confirmation kém tin cậy hoặc bị stall	Tăng committee size; coi micro chỉ là soft-confirmation; hard-finality vẫn cần macro quorum toàn cục
Equivocation	Tạo fork hoặc phá accountable safety	Slash rõ ràng cho double-vote, surrounding vote, conflicting checkpoint sign
MEV/short reorg	Censor, fee sniping, reorder	Không coi micro là settlement; macro-finality ngắn hơn Gasper; phí aged-inclusion để giảm động cơ trì hoãn

Tấn công	Hậu quả	Đối sách trong LUA-DAG
Spam DAG/storage blowup	Phình bộ nhớ và băng thông	Batch bond, fee floor, giới hạn bytes/slot, garbage collection sau finality
Client monoculture	Lỗi phần mềm thành lỗi hệ thống	Khuyến nghị đa client, fuzzing, TLA+, testnet adversarial

Các đối sách này đều có tiền lệ mạnh trong literature và implementation hiện hành: long-range/weak subjectivity ở Ethereum PoS; equivocation/slashing ở Casper/Gasper/Ethereum; reliable dissemination ở Narwhal; và collector/leader failover ở HotStuff/Tendermint class protocols. [37]

Một điểm quan trọng của LUA-DAG là **không trộn lẫn bảo đảm**. Người dùng, bridge và exchange chỉ nên coi giao dịch “không thể đảo” khi đã nằm dưới macro-finality. Soft-confirmation vẫn hữu ích, nhưng không nên bị marketing thành “instant finality” theo nghĩa nghiêm ngặt. Bản thân lộ trình single-slot finality của Ethereum tồn tại vì cộng đồng này cũng nhận ra khoảng cách giữa “head accepted nhanh” và “block finalized an toàn”. [38]

Kế hoạch đánh giá hiệu năng

Khung benchmark và tham số mô phỏng

Kế hoạch đánh giá phải tách riêng **consensus cost**, **data-plane cost** và **execution cost**. Nếu không tách, kết quả sẽ lẫn lộn giống nhiều benchmark blockchain cũ: TPS nhìn cao nhưng thực ra né execution, hoặc latency nhìn thấp nhưng payload gần như rỗng. Narwhal/Tusk và HotStuff đều cho thấy payload size, batching, topology WAN/LAN và số validator làm thay đổi kết quả rất mạnh; Ethereum SSF research còn chỉ ra rằng signature aggregation có thể trở thành bottleneck khác với verification thuần. [39]

Một kế hoạch benchmark nghiêm túc cho LUA-DAG nên có tối thiểu ma trận sau:

Nhóm tham số	Giá trị nên quét
Số validator	25, 50, 75, 100, 150
Mô hình mạng	cùng datacenter; 5 region WAN; 8 region WAN; burst delay/packet loss trước GST
Tải giao dịch	payment-only; mixed token transfer; smart-contract state update; read-heavy
Kích thước tx	250B, 512B, 1KB, 2KB
Kích thước batch	256KB, 512KB, 1MB
Cỡ micro committee	128, 256, 512, 1024
Cửa sổ macro w	2, 4, 8 micro-slots
Lỗi và tấn công	0 faults; 1 faulty collector; f Byzantine validators; data withholding; equivocation; partition ngắn

Các chỉ số đo phải gồm: TPS nộp vào, TPS soft-confirmed, TPS finalized; latency p50/p95/p99 cho soft-confirmation và hard-finality; bandwidth/node; CPU verify/aggregate signatures; disk growth của DAG; thời gian state sync từ checkpoint; thời gian khôi phục sau partition; tỉ lệ batch bị challenge; fairness của inclusion delay; và rollback depth tối đa của soft-confirmation. Những chỉ số này phản ánh trực tiếp các trade-off được survey và protocol papers nhấn mạnh, thay vì chỉ tối đa hóa một con số TPS duy nhất. [40]

Chuẩn so sánh và kết quả kỳ vọng

Chuẩn so sánh hợp lý nhất cho LUA-DAG là bốn tuyến: HotStuff/Tendermint class BFT chain; Narwhal-HotStuff; Tusk/Bullshark; và một finality gadget kiểu Gasper cho semantics hard-finality. Nếu benchmark muốn công bằng, nên so trên cùng loại payload, cùng topology WAN, cùng batch size, cùng execution off/on. Bullshark paper, chẳng hạn, so WAN 10/20/50 parties với 500KB maximum block size và 512B transaction size; đây là một baseline rất hợp lý để tái sử dụng. [41]

Dựa trên các benchmark đã công bố của Narwhal/Tusk/Bullshark, một **giả thuyết** hợp lý cho LUA-DAG là nó nên giữ được phần lớn throughput của DAG fast path, nhưng trả thêm một premium latency vừa phải ở hard-finality vì phải chờ macro checkpoint. Nói cách khác, nếu Bullshark đạt khoảng 125k TPS ở 2 giây và Tusk khoảng 160k TPS ở 3 giây trong điều kiện đã nêu, thì LUA-DAG hợp lý nhất khi nhắm **soft-confirmation gần vùng Bullshark/Tusk**, còn **hard-finality chậm hơn soft path nhưng vẫn chỉ vài giây thay vì hàng phút**. Đây là kỳ vọng suy luận từ structure của giao thức, không phải kết quả đã chứng minh thực nghiệm. [42]

Biểu đồ dưới đây biểu diễn **kỳ vọng giả thuyết** cho throughput khi thay đổi số validator, với giả định mô phỏng gần benchmark Bullshark: 512B/tx, batch 500KB, nhiều region WAN, $W = 4$. Thanh biểu diễn TPS đã hard-finalized; đường biểu diễn TPS soft-confirmed.

xychart-beta

```
title "Kỳ vọng giả thuyết về throughput của LUA-DAG"
x-axis [25, 50, 75, 100, 150]
y-axis "TPS" 0 --> 140000
bar [115000, 100000, 90000, 78000, 58000]
line [135000, 120000, 108000, 92000, 65000]
```

Với giả thuyết trên, ba bottleneck nhiều khả năng sẽ nổi lên trước: **(i)** availability dissemination và I/O lưu batch; **(ii)** aggregate signatures ở macro layer; và **(iii)** execution/state storage nếu workload có smart-contract logic nặng. Narwhal/Tusk đã chỉ ra dissemination là nút thắt thực sự ở nhiều thiết kế; còn Ethereum SSF research chỉ ra rằng khi finality tăng tốc, bottleneck dịch dần sang aggregation/networking thay vì pure verification. Vì vậy, nếu prototype LUA-DAG “thua kỳ vọng”, điều cần kiểm tra đầu tiên nhiều khả năng không nằm ở logic safety mà ở network stack, aggregation pipeline hoặc state engine. [43]

Kế hoạch mô phỏng và formal validation

Ngoài benchmark thực, LUA-DAG nên có một simulator rời rạc để quét space tham số rộng hơn production testnet khó tái tạo: committee size, fraction Byzantine theo stake, bursty delay,

correlated outages, churn, collector DoS và batch withholding. Mục tiêu của simulator không phải tạo TPS đẹp, mà là đo: xác suất thiếu MicroQC trong cửa sổ thời gian nhất định; xác suất stall hard-finality; throughput degradation dưới faults; và độ nhạy của protocol với w , Δ , batch size, GC horizon. Asynchronous BFT literature như HoneyBadger và DAG-Rider là chuẩn tham chiếu tốt cho phần “điều gì xảy ra khi abandon partial synchrony assumptions trong một thời gian”. [44]

Lộ trình triển khai, nguồn lực và rủi ro

Lộ trình kỹ thuật đề xuất

Giai đoạn	Mục tiêu chính	Kết quả đầu ra	Nhân sự ước lượng	Thời lượng ước lượng
Spec và threat model	Đặc tả protocol, message types, invariants, slash conditions	Whitepaper kỹ thuật + pseudo-spec + test vectors	2 protocol engineers + 1 security researcher	6–8 tuần
Mô hình hình thức	TLA+/Ivy/PlusCal cho safety/liveness skeleton	Model checked invariants, counterexample traces	1 formal methods engineer + 1 protocol engineer	6–10 tuần
Prototype availability DAG	Batching, erasure coding, DAG storage, GC, challenge/serve	Rust/Go prototype data plane	3 distributed systems engineers	8–12 tuần
Prototype micro-ordering	VRF selection, collectors, MicroQC, failover	Benchmarkable fast path	2 protocol engineers + 1 performance engineer	8–10 tuần
Prototype macro-finality	Macro checkpoint, aggregate signatures, slashing evidence, checkpoint sync	Hard-finality chain + header proofs	2 protocol engineers + 1 cryptography engineer	8–12 tuần
State sync và light client	Checkpoint sync, snapshot proofs, mobile verifier	CLI/node sync + light-client SDK	2 client engineers	6–8 tuần
Adversarial testnet	Fault injection, partitions, recovery, metrics	Public report + tuning recommendations	2 SRE/perf engineers + toàn đội hỗ trợ	8–12 tuần
Audit và hardening	External review, fuzzing, chaos tests	Audit report, remediation log	external auditors + 1 internal security lead	8–10 tuần

Nếu làm nghiêm túc, một nguyên mẫu nghiên cứu đủ sức benchmark cần khoảng **6–8 người nòng cốt** trong **12–18 tháng**, tương đương cỡ **70–100 person-months**. Nếu thêm execution engine song song, prover/STARK integration, hay production validator operations, con số sẽ tăng đáng kể. Đây là **ước lượng triển khai** của báo cáo, không phải số lấy từ nguồn bên ngoài.

Rủi ro chủ đạo và giảm thiểu

Rủi ro	Vì sao nghiêm trọng	Cách giảm thiểu
Độ phức tạp thiết kế	DAG + micro + macro dễ tạo bug liên tầng	Giữ từng layer tối giản; model-check invariants; codegen từ spec
Semantics soft vs hard bị hiểu nhầm	Ứng dụng dùng nhầm soft-confirm như hard-finality	API tách bạch accepted, soft_confirmed, finalized; docs bắt buộc
Aggregation bottleneck	Macro layer có thể nghẽn trước cả ordering	Pipeline hóa BLS aggregation, benchmark subnet aggregation sớm
Storage blowup và GC sai	DAG có thể ngốn disk và gây data loss	GC chỉ theo finalized horizon + retrieval challenge tests
Incentive lệch	Validators bỏ bê availability nhưng vẫn sẵn macro reward	Reward phân rõ cho availability duty; challenge-based penalties
Weak subjectivity vận hành kém	Node mới sync sai checkpoint	Multi-source checkpoint pinning; client UX xác thực chéo
Tấn công collector chọn mục tiêu	Làm tăng latency mềm	Private VRF sortition, ranked backups, aggressive failover
Monoculture client	Một lỗi logic thành consensus failure	Tối thiểu hai codebase độc lập trước public mainnet

Rủi ro lớn nhất ở đây không phải “safety theorem sai” theo kiểu huyền hoặc, mà là **sai ranh giới module**: khi availability, ordering, state execution và light-client proofs được ghép với nhau không thật sạch, nhiều bug hạng nặng sẽ lộ ra ở phần chứng cứ và khôi phục sau fault. Đây cũng là lý do lộ trình nên đi từ **payment-style VM tối giản** rồi mới mở rộng sang execution phức tạp. Narwhal/Tusk tự thân cũng thừa nhận execution engine là phần được để lại cho bước sau; đúng ra một consensus mới phải học bài học này sớm thay vì benchmark bằng workload quá hiền. [45]

Đánh giá cuối cùng là: nếu mục tiêu là một L1 hoặc settlement layer **vừa permissionless, vừa tiết kiệm năng lượng, vừa có đường lên hàng chục-ngàn đến trăm-ngàn TPS**, thì hướng đáng đầu tư nhất hiện nay không phải phát minh thêm một “leader election kỳ lạ” đơn lẻ, mà là **kết hợp availability DAG với finality có trách nhiệm giải trình**. LUA-DAG là một bản thiết kế mới theo đúng logic đó: **dùng DAG để không nghẹt dữ liệu, dùng committee ngẫu nhiên để có head nhanh, và dùng macro checkpoint/slashing để có finality thật**. Nếu triển khai đúng, nó có triển vọng lấp khoảng trống giữa throughput của Bullshark/Tusk và semantics finality/bridge-friendliness của Casper/Gasper. [16]

[1] [7] <https://ethereum.org/developers/docs/consensus-mechanisms/>
<https://ethereum.org/developers/docs/consensus-mechanisms/>

[2] [21] [32] [37] Weak subjectivity | ethereum.org
https://ethereum.org/developers/docs/consensus-mechanisms/pos/weak-subjectivity/?utm_source=chatgpt.com

[3] [9] [33] <https://bitcoin.org/bitcoin.pdf>
<https://bitcoin.org/bitcoin.pdf>

[4] [12] [16] [18] [23] [24] [34] [39] [43] [45] <https://aptoslabs.com/pdf/2105.11827.pdf>
<https://aptoslabs.com/pdf/2105.11827.pdf>

[5] [19] <https://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf>
<https://groups.csail.mit.edu/tds/papers/Lynch/jacm88.pdf>

[6] [17] [28] https://drproduck.github.io/paper/gasper/gasper_final.pdf
https://drproduck.github.io/paper/gasper/gasper_final.pdf

[8] [40] <https://arxiv.org/abs/1904.04098>
<https://arxiv.org/abs/1904.04098>

[10] [25] [36] <https://arxiv.org/pdf/1803.05069>
<https://arxiv.org/pdf/1803.05069>

[11] <https://dl.acm.org/doi/10.1145/3579845>
<https://dl.acm.org/doi/10.1145/3579845>

[13] [38] <https://ethereum.org/roadmap/single-slot-finality/>
<https://ethereum.org/roadmap/single-slot-finality/>

[14] <https://ethereum.org/developers/docs/nodes-and-clients/light-clients/>
<https://ethereum.org/developers/docs/nodes-and-clients/light-clients/>

[15] <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
<https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>

[20] <https://cryptochainuni.com/wp-content/uploads/Ouroboros-Praos-An-adaptively-secure-semi-synchronous-proof-of-stake-blockchain.pdf>

<https://cryptochainuni.com/wp-content/uploads/Ouroboros-Praos-An-adaptively-secure-semi-synchronous-proof-of-stake-blockchain.pdf>

[22] <https://arxiv.org/pdf/1810.08092>

<https://arxiv.org/pdf/1810.08092>

[26] [41] [42] <https://aptoslabs.com/pdf/2201.05677.pdf>

<https://aptoslabs.com/pdf/2201.05677.pdf>

[27] <https://arxiv.org/pdf/1710.09437>

<https://arxiv.org/pdf/1710.09437>

[29] [31] <https://ethereum.org/developers/docs/consensus-mechanisms/pos/>

<https://ethereum.org/developers/docs/consensus-mechanisms/pos/>

[30] <https://ethereum.org/developers/docs/consensus-mechanisms/pos/rewards-and-penalties/>

<https://ethereum.org/developers/docs/consensus-mechanisms/pos/rewards-and-penalties/>

[35] Ethereum proof-of-stake attack and defense | ethereum.org

https://ethereum.org/developers/docs/consensus-mechanisms/pos/attack-and-defense?utm_source=chatgpt.com

[44] <https://eprint.iacr.org/2016/199>

<https://eprint.iacr.org/2016/199>